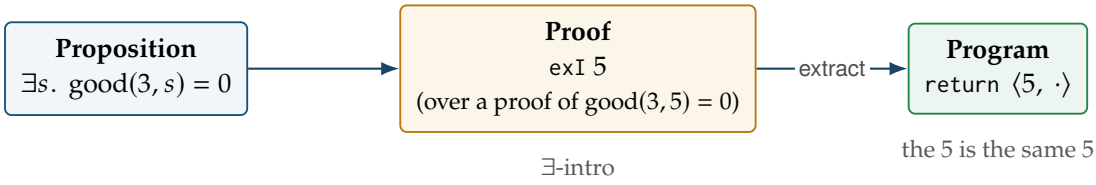
One construction, two readings

the proof is *the program — extraction is the map that shows it*

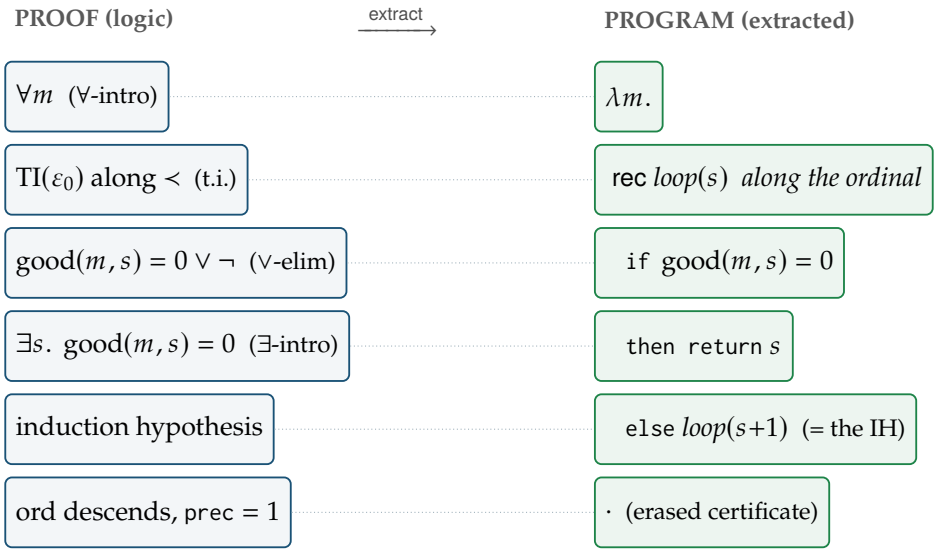
Modified realizability is the Curry–Howard correspondence with the map made explicit: a proof (a `Deriv`) and its extracted program (a realizer) are distinct objects joined by extract, and extract is *structure-preserving* — it sends each proof rule to one program operation. The dictionary, and then the same dictionary at work on two theorems.

Proposition / proof rule	Program operation	as a type
∀-intro / ∀-elim	$\lambda k.\_$ / apply at a value	Π (dependent function)
→-intro / →-elim	$\lambda h.\_$ / apply	function
∃-intro / ∃-elim	`return` $\langle w, \cdot \rangle$ / `let` $\langle w, p \rangle$	Σ (dependent pair)
∧-intro / ∨-elim	pair $\langle \_, \_ \rangle$ / `case`	product / sum
induction	fold-recursion	Nat.rec
transfinite induction TI(ε_0)	well-founded recursion along $\prec$	WellFounded.fix
hypothesis / induction hyp.	variable / *recursive call*	bound variable
equation, ⊥	· (erased — no runtime content)	proof-irrelevant

Reading 1 — witness supplied by hand ⇒ a constant. The proof of $\exists s.\ \mathrm{good}(3, s) = 0$ *contains* the answer; extraction reads it out.



Reading 2 — witness produced by TI(ε_0) ⇒ a recursion. The proof of $\forall m\ \exists s.\ \mathrm{good}(m, s) = 0$ contains no literal witness — it contains a *recursion*, and extraction turns the recursion into a program that *computes* the stopping step. Left, the proof; right, the extracted program; each row is one rule of the dictionary. (This is the exact output of `#realizerCH goodsteinTheorem`, abbreviated to the spine.)



The same 29-node object is a proof that every Goodstein sequence terminates (`#print goodsteinTheorem`*) and a program that computes when (*`#realizer goodsteinTheorem`*). The shape of the induction is the shape of the recursion: witness-by-hand gives a constant, witness-by-*$\mathrm{TI}(\varepsilon_0)$ *gives an* ε_0*-recursion. The induction hypothesis* is *the recursive call; the ordinal descent is the one part with no runtime content.*